¿Hay vida más allá de Ubuntu?

Guía de supervivencia para desarrollar en ROS 2 desde tu sistema favorito

Francisco Miguel Moreno

franciscom.moreno@urjc.es

Intelligent Robotics Lab · Universidad Rey Juan Carlos

Intelligent Robotics Lab

Universidad Rey Juan Carlos

ROSCon España 2026 · Valencia



Plataforma	**REP-2000**
Ubuntu	Tier 1
Windows	Tier 1
RHEL	Tier 2
macOS, Debian	Tier 3
Otras distros...	no aparecen

- Fuera de Ubuntu, en la práctica casi nada funciona a la primera.
- Y en Ubuntu, `apt` solo tiene **la última versión** publicada: reinstalar dentro de 6 meses te da otro ROS.
- Aunque RHEL es Tier 2, sus RPM no sirven en Fedora.

¿Qué opciones hay para **desarrollar** en ROS 2 fuera de Ubuntu?

REP-2000, plataformas de Kilted (Rolling apunta a las mismas).

¿Quién se sale del camino recomendado?

Javi

Usa "Arch, BTW". Compila su kernel y lo quiere todo en la última versión.

una distro que ROS ni menciona

Sara

Empresa grande. Por seguridad, Linux prohibido y Windows sin admin.

Windows sin admin ni VMs

Lucía

Viene del desarrollo web: `npm install` y a funcionar. Su Mac es su herramienta.

macOS; quiere algo tipo npm

Charlie

Usa Bluefin, una Fedora inmutable: el sistema base es de solo lectura.

nada se instala en el sistema

Las opciones

En tu sistema

- **apt** en Ubuntu
- **Desde fuentes** en tu distro

Contenedores

- **Docker / Podman** + devcontainer
- **Distrobox**
- **Fuentes** dentro del contenedor

Gestores de paquetes

- **Pixi** (RoboStack)
- **Conan**
- **Nix** (nix-ros-overlay)

El camino clásico / recomendado

apt en Ubuntu

- El camino documentado y probado por la buildfarm.
- Cero compilación de ROS.
- Pero: una Ubuntu concreta por distro de ROS, y sin fijar versiones.

Desde fuentes en tu sistema

Ejemplo en Fedora 44:

- 150 paquetes de ROS 2 (solo lo necesario).
- Múltiples dependencias de `dnf`; rosdep con reglas incompletas.
- Python 3.14, GCC 16, CMake 4.3: más nuevos que cualquier Tier 1.

Posible, pero no recomendado.

Contenedores: Ubuntu, pero dentro de una caja

Docker / Podman + devcontainer

- Ubuntu + ROS + `rosdep`, en una imagen.
- ROS desde apt o compilado desde fuentes.
- El código se monta desde el host.
- Windows y macOS, vía VM (Docker Desktop).

Imagen 5,3 GB · build 2,5 min / 15 min

Distrobox

```
distrobox assemble create
distrobox enter easynav-rolling
rosdep install --from-paths src -y
colcon build
```

- Mismo usuario, `$HOME`, red, GUI y USB.
- Solo Linux.

Pixi: un lockfile, tres sistemas operativos

```
[workspace]
channels = ["robostack-rolling", "conda-forge"]
platforms = ["linux-64", "osx-arm64", "win-64"]

[dependencies]
ros-rolling-rclcpp = "*"

[tasks]
build = "colcon build"
```

```
pixi install && pixi run demo
```

- Sin root, sin tocar el sistema.
- ROS 2 como paquetes conda (RoboStack).
- `pixi.lock` para las tres plataformas.
- 1,5 min de setup.

Workshop de Pixi + ROS 2:
`github.com/IntelligentRoboticsLabs/workshop-pixi`

Conan: ROS 2 como una dependencia C++ más

```
[requires]
ros-kilted/2026.06.17

[options]
ros-kilted/*:variant=desktop
```

- ROS 2 en tu proyecto C++, igual que Boost u OpenCV.
- Sin `rosdep` ni `apt`, en Linux, macOS y Windows.
- Binarios compartibles en el servidor de la empresa.

Instalación ∼15 min · build 2 min

Un proyecto en maduración

- `ros-conan`: todo ROS 2 en **una única receta**, con variantes (`core`, `base`, `desktop`…).
- Por ahora, solo **kilted**.
- Lo que no está en la receta, al workspace.

Comparativa

	apt	**Fuentes (Docker)**	**Contenedor**	**Distrobox**	**Pixi**	**Conan**
Sistemas	Ubuntu	todos $*^1$	todos $*^1$	Linux	todos	todos
Sin root	×	✓	✓	✓	✓	✓
Fija versiones	×	✓ `.repos`	× (apt)	× (apt)	✓ lock	✓ lock
Integración con el host	nativa	baja	baja	alta	alta	alta

$*^1$ En macOS y Windows, a través de una VM (Docker Desktop, WSL2).
$*^2$ Build en conan compilando ROS 2 de fuente.

¿Hay vida más allá de Ubuntu?

Demos, scripts y slides:
`https://github.com/butakus/roscon26_talk`

Francisco Miguel Moreno
`franciscom.moreno@urjc.es`

